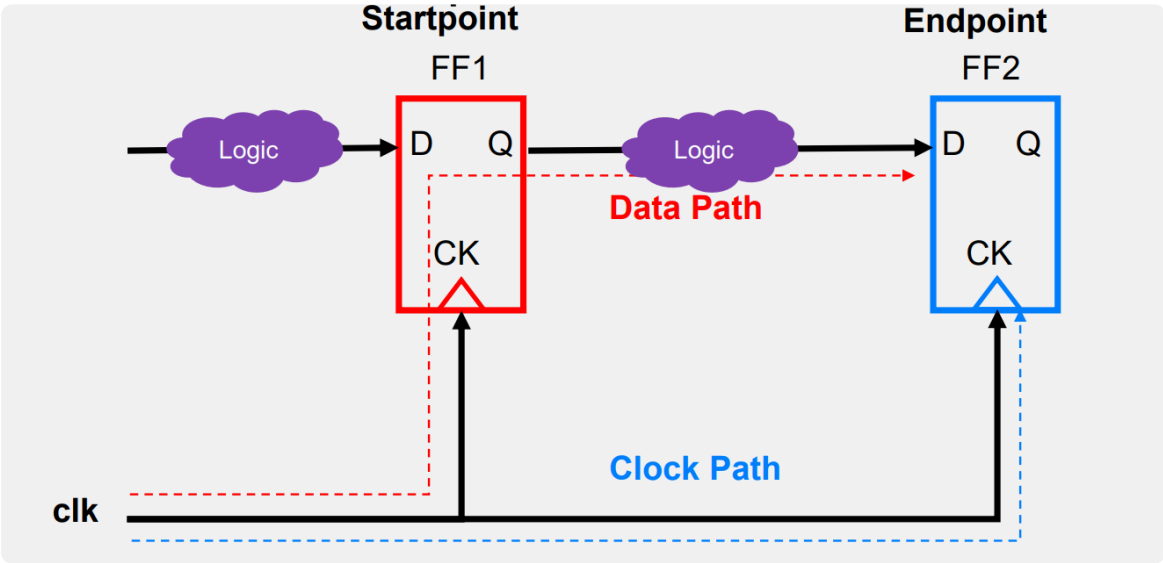
NYCU PDA Lab3 Optimizer and Legalizer Co-optimization

Table of Content

- NYCU PDA Lab3 Optimizer and Legalizer Co-optimization
  - Preface
  - Introduction
  - Problem Formulation
  - Problem Objective
  - Input Format
  - Output Format
  - Runtime Factor
  - Program Requirements
  - Report
  - Submission
  - Evaluator
  - Grading
  - Disclaimer
  - Visualizer
  - Release
  - Reference
  - Q&A
  - Contacts

Preface

- 1. Timing ECO

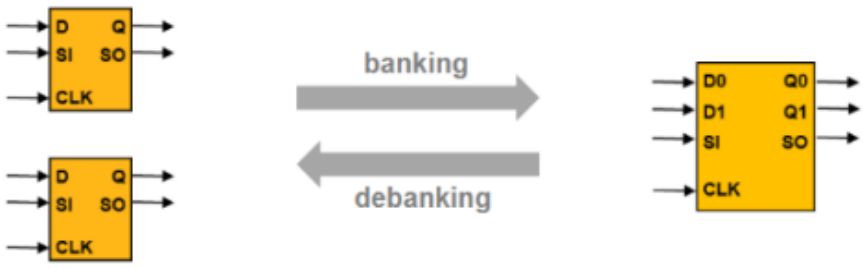


reference:How to ECO timing? (https://www.synopsys.com/content/dam/synopsys/implementation&signoff/white-papers/physical-aware-wp.pdf)

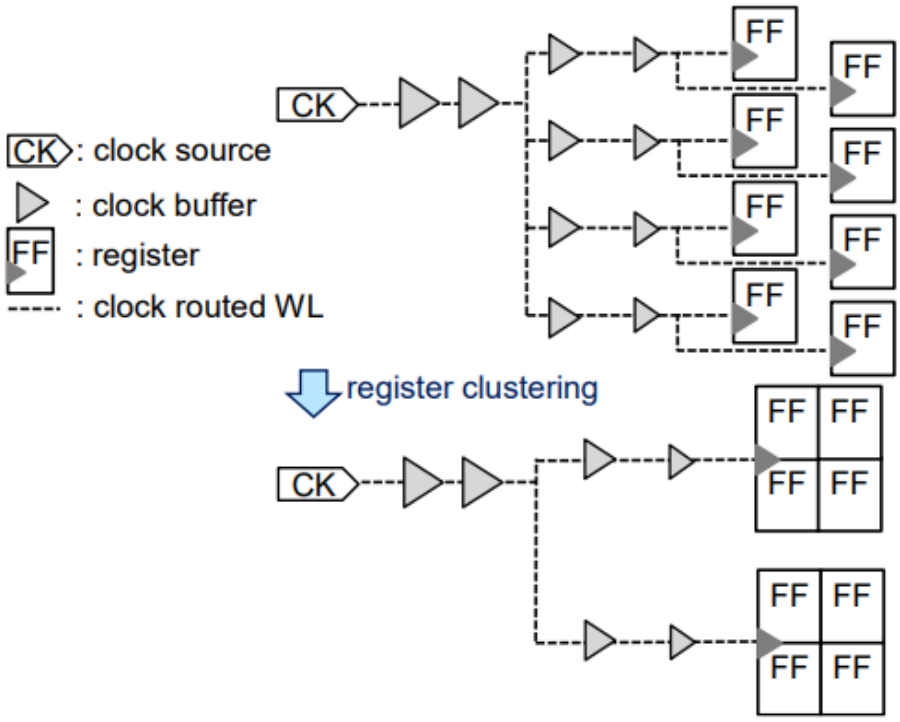
In the timing ECO flow, or during hold/setup time optimization, additional cells may be inserted into the legalized placement. For example, to fix setup time, a clock buffer can be inserted into the clock path, while a hold buffer can be added to the data path to address hold time issues. Another method to optimize timing is through cell sizing. However, these optimization methods may result in illegal placement, causing overlaps between cells.

- 2. Power and Timing Optimization Using Multibit Flip-Flop

reference:2024 ICCAD Problem B (https://drive.google.com/file/d/1k2ssyOYBHmjz8so-Elq6IMcfIC33SLVx/view)



Another optimization method in place-and-route EDA tools is Flip-Flop banking. By merging two 1-bit flip-flops into a 2-bit flip-flop, power consumption and area can be significantly improved. However, the total negative slack (TNS) should also be considered as a potential drawback.



reference:Graceful Register Clustering by Effective Mean Shift Algorithm for Power and Timing Balancing (https://waynelin567.github.io/files/meanshift.pdf)

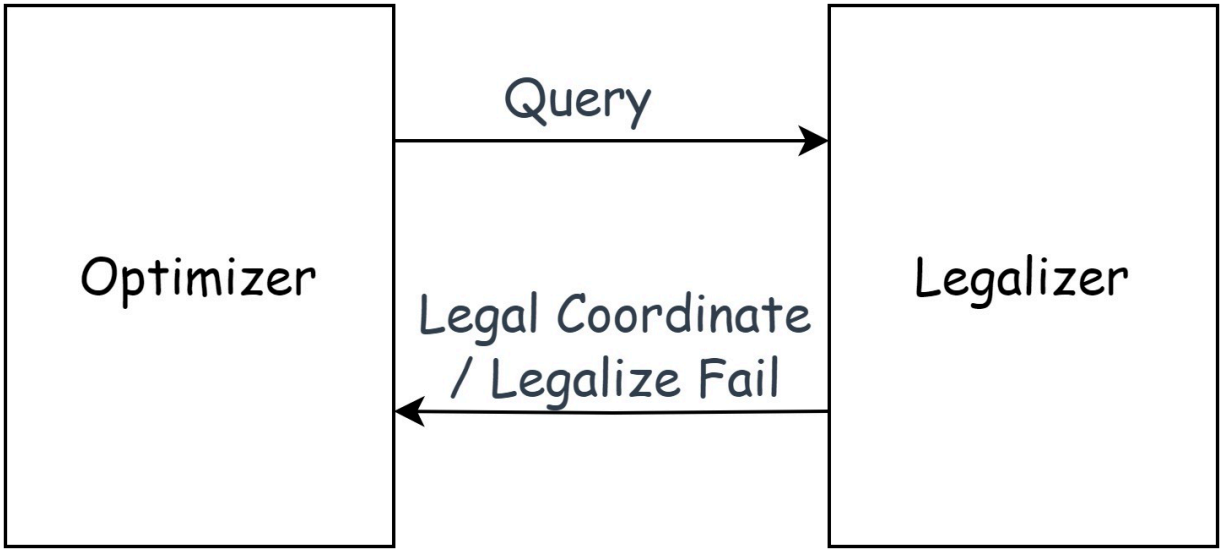
Flip-flop banking offers several advantages in electronic design automation tools, especially for place-and-route optimization:

1. Reduced Power Consumption: By merging multiple 1-bit flip-flops into a single multi-bit flip-flop (e.g., 2-bit, 4-bit), the number of clock drivers and interconnections is reduced, which lowers dynamic power consumption.
2. Smaller Area: A multi-bit flip-flop requires less silicon area compared to multiple single-bit flip-flops, reducing overall chip area and allowing more efficient use of space.
3. Simplified Clock Distribution: Fewer clock trees are needed to drive multi-bit flip-flops, which can reduce clock skew and power dissipation.
   However, it's essential to balance these benefits with potential drawbacks like increased total negative slack (TNS), which might require further timing optimization.

Introduction

As mentioned earlier, many optimization techniques can result in cell overlap, leading to illegal placement. Therefore, a fast algorithm is needed to perform local legalization rather than full-chip legalization and determine whether the optimization is feasible at that moment. Consequently, a quick API for the legalizer is necessary so that the optimizer can frequently check whether the optimization is easy to legalize.

For example:



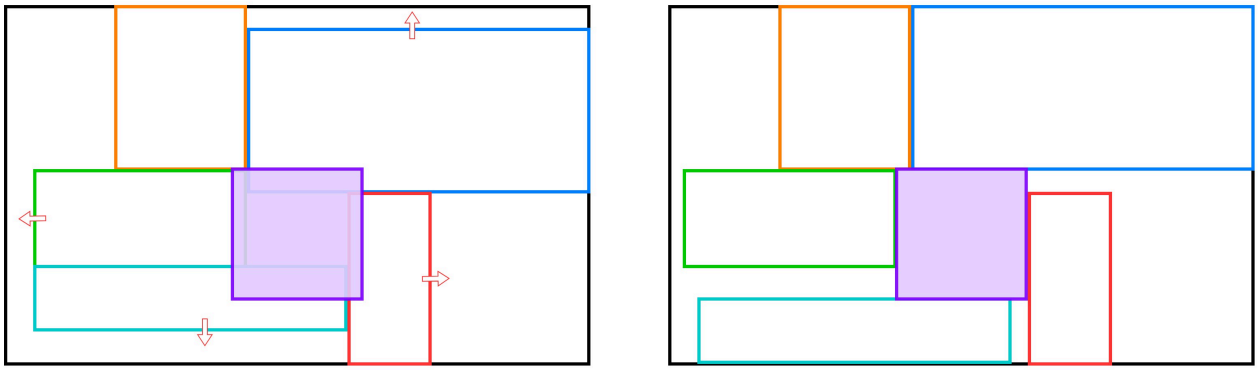
Problem Formulation

Given a legalized placement P and a set of cells C to be inserted into the placement, implement an efficient **local** legalizer that legalizes the cells in C into P one by one. The local legalizer should aim to:

- Maintain the legality of the placement after each insertion.
- Minimize the disturbance to the existing placement.
- Ensure efficient execution, particularly for large instances of C.

Focus on designing a method that prioritizes local adjustments during the legalization process.

For example:



Problem Objective

To avoid full-chip legalization using well-known algorithms like Tetris Legalization [Hill, Patent 2002] (https://patentimages.storage.googleapis.com/0d/17/0f/8b9dbb9343b70f/US6370673.pdf) or Abacus Legalization [Spindler et al., ISPD'08] (https://dl.acm.org/doi/pdf/10.1145/1353629.1353640),

a special weighted cost function is implemented to evaluate the quality of your results.

Given a legalized placement P and a set of cells C to be inserted into the placement.

$$cost = (\sum_{i=1}^{|C|} \alpha \cdot x_i) + \beta \cdot y$$

NYCU PDA L... Try HackMD (https://hackmd.io?utm_source=view-page&utm_medium=logo-nav)

where

$$\begin{cases} |C| & = & cardinality\ of\ set\ C \\ \alpha & = & parameter1 \\ x_i & = & \#\ cell\ move\ in\ P \\ \beta & = & parameter2 \\ y & = & total\ displacement\ from\ initial\ position(ManhatanDistance) \end{cases}$$

Given a legalized placement `P` and a set of cells `C` to be inserted into the placement, implement an efficient **local** legalizer that legalizes the cells in `C` into `P` one by one. Specifically:

- After legalizing each cell `cell_i` from `C`, it should be added to the placement `P`.
- The updated placement `P` after adding `cell_i` should maintain legality, i.e., `P <= P + cell_i`.
- The process should continue for each cell in `C` until all cells are legalized.

The goal is to minimize disturbance to the existing placement and ensure efficient execution, especially for large sets of cells in `C`.

Input Format

Disclaimer: Most of the testcase will be modified from 2024 ICCAD Problem B (https://drive.google.com/file/d/1k2ssyOYBHmjz8so-Elq6IMcfIC33SLVx/view). Thanks for (Synopsys, Inc) to release the benchmark.

In this section we will define each syntax for data input. Most of the syntaxes are the floating point numbers and we expect students to take care of the data range within DBL_MAX.

- 1. Weight factors of the cost metrixs: α, β values are given out as Alpha, Beta, respectively.

```
Syntax
    Alpha <alphaValue>
    Beta <betaValue>

Example
    Alpha 100
    Beta 200
```

- 2. DieSize describes the dimension of the die, namely the placement area of the design.

```
Syntax
    DieSize <lowerLeftX> <lowerLeftY> <upperRightX> <upperRightY>

Example
    DieSize 0 0 50 30
```

- 3. Cell attribute and legalized coordinate. For fix cell, you are not allowed to move when you try to legalize the placement later.

```
Syntax
    <cellName> <lowerLeftX> <lowerLeftY> <width> <height> <FIX/NOTFIX>

Example
    FF_1_0 8 0 5 10 NOTFIX
    FF_1_1 14 0 5 10 NOTFIX
    FF_1_2 14 20 5 10 NOTFIX
    C4 10 10 5 10 FIX
```

- 4. Placement rows. The given row would start from (<startX>, <startY>) and has a <totalNumOfSites> of cell sites repetitively placed back-to-back in x direction. A cell site is a small rectangle defined in PlacementRows with a dimension of (<siteWidth>, <siteHeight>). The leftmost coordinate of the PlacementRows is <startX> and the rightmmost coordinate of the PlacementRow is <startX + siteWidth x totalNumOfSites>. This syntax defines the placement sites in the design. The height of PlacementRow is standard cell site height, and we will also give the site count, which means that the height of PlacementRow is the height of a cell site and the width of. Every cell should be placed on the PlacementRows, with its lower left corner aligning to the site grid of the PlacementRows.

Note that to simplify the problem, the siteWidth should always be 1 in this assignment. This means the minimum precision of the cell coordinates should be at least 1 unit in the placement.

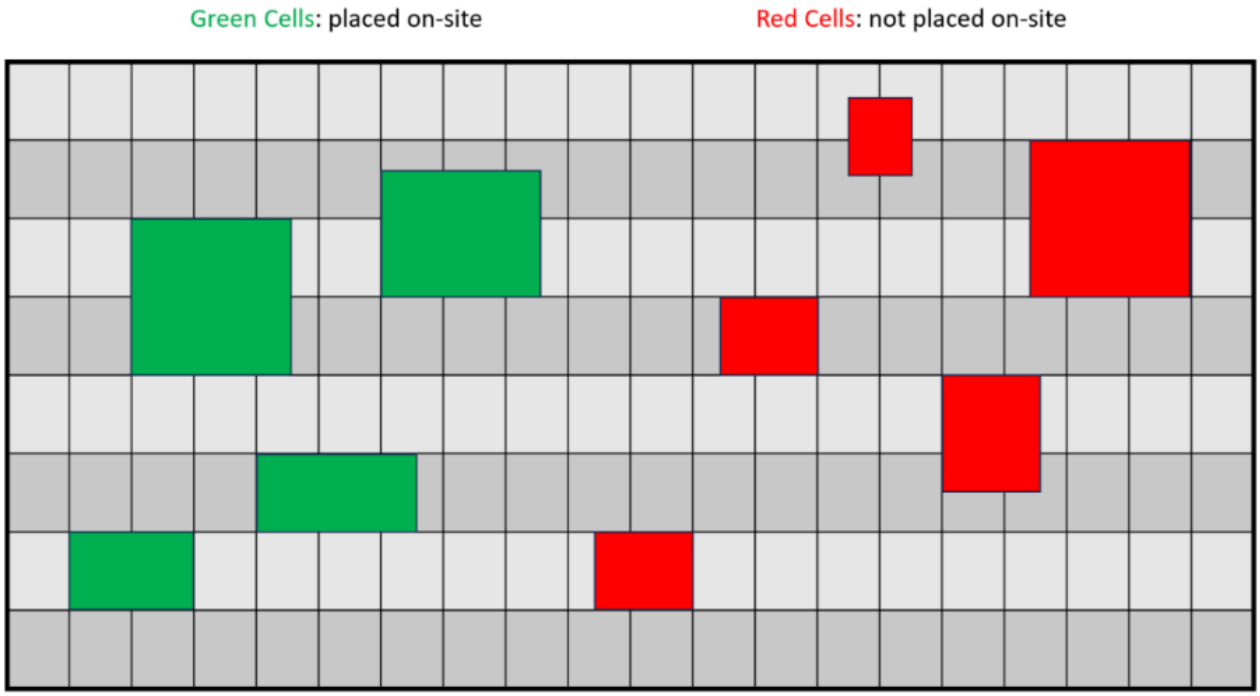


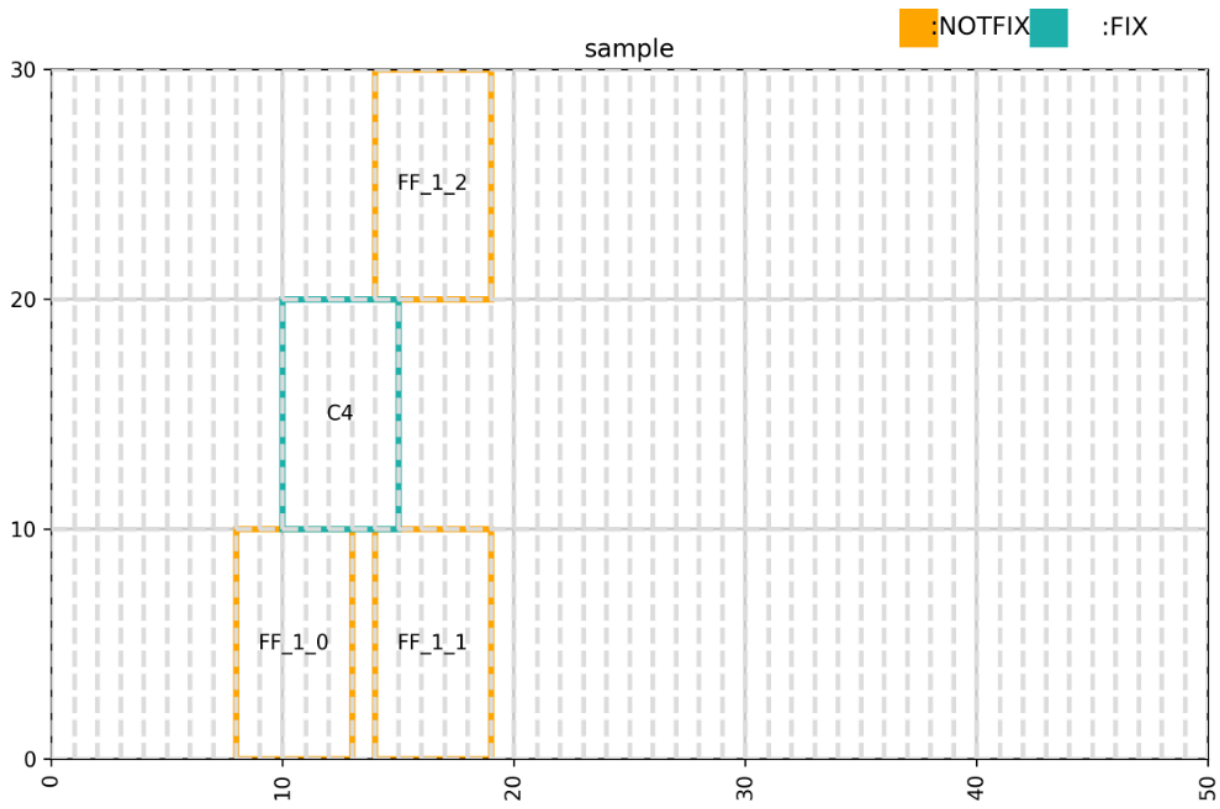
Figure 4 in 2024 ICCAD Problem B (https://drive.google.com/file/d/1k2ssyOYBHmjz8so-Elq6IMcfIC33SLVx/view). An example to illustrate what placing on-site means. We define on-site as cells with their bottom-left corner aligning to the site grid given from the PlacementRows from the design data input. The 4 green cells on the left part of the figure are placed on-site as their bottom-left corners are aligning with the site grids of the PlacementRows; the 5 red cells on the right side of the figure are not placed on-site as none of the cells have their bottom-left corner placed on the site grid.

```
Syntax
    PlacementRows <startX> <startY> <siteWidth> <siteHeight> <totalNumOfSites>

Example
    PlacementRows 0 0 1 10 10
```

- Example

```
Alpha 100
Beta 200
DieSize 0 0 50 30
FF_1_0 8 0 5 10 NOTFIX
FF_1_1 14 0 5 10 NOTFIX
FF_1_2 14 20 5 10 NOTFIX
C4 10 10 5 10 FIX
PlacementRows 0 0 1 10 50
PlacementRows 0 10 1 10 50
PlacementRows 0 20 1 10 50
```



- 5. Optimize Step

  Finally, here is the optimizer output, which aims to merge multiple flip-flops into multi-bit flip-flops. Students are expected to remove the list of flip-flops to be banked in the legalized placement P, and add the new multi-bit flip-flop into the legalized placement P without overlapping other cells, while also minimizing the given cost function.

  ```
  Syntax
      Banking_Cell: <list of ff to bank> --> <merged ff name> <x> <y> <w> <h>
  Example
      Banking_Cell: FF_1_19425 FF_1_16134 --> FF_2_0 732360 634200 9180 4200
  ```

  Please note that the merged cell coordinates might not be legalized. Find a space to place them on the placement row without overlapping with other cells.

In conclusion, there are two files per testcase: the legalized placement information will be stored in the *.lg file, and the optimizer step will be stored in the *.opt file.

Output Format

For each optimization step, please output the legalized cell for the given merged flip-flop (FF), as well as all the cells that were moved during the legalization of this merged FF.

```
Syntax
    <merged ff x> <merged ff y>
    <num of moved cell>
    <cel1_name> <cell1_x> <cell1_y>
Example
    734500 634200
    2
    FF_1_0 725000 623000
    FF_1_1 742000 695000
```

Runtime Factor

Runtime limit is 30 minutes for each case in our server. The hidden cases will be in the same scale as public cases. We would like to introduce the runtime factor in this homework to encourage the ideas with faster turnaround-time.

$$runtime\ factor = 0.02 \cdot \log_2(\frac{elapse\ time\ of\ test\ binary}{median\ elapse\ time})$$

$$runtime\ factor\ bounded = max(-0.1, min(0.1, runtime\ factor))$$

$$Performance\ score = PR(\ Initial\ score \cdot (1 + runtime\ factor\ bounded))$$

Program Requirements

Your program should be executed by:

```
$./Legalizer *.lg *.opt *_post.lg
```

After program finish, we expect a *_post.lg to show all info required for each optimized step.

For example:

```
$./Legalizer testcase1.lg testcase1.opt testcase1_post.lg
```

Report

Please write a quick report explaining how your algorithm works, including:

- Pseudocode
- Time complexity analysis
- Special features of your program

Submission

```
$ tar cvf studentID.tar studentID
```

We expect your file can be executed after

```
$ tar xvf studentID.tar
$ cd studentID
$ ls --> should show Makefile, studentID_rpt.pdf and your src
$ make -j --> compile the binary with static linking
$./Legalizer *.lg *.opt *_post.lg
```

Please submit your tar file to e3 by 11/15

Stackoverflow static linking: https://stackoverflow.com/questions/26103966/how-can-i-statically-link-standard-library-to-my-c-program
(https://stackoverflow.com/questions/26103966/how-can-i-statically-link-standard-library-to-my-c-program)

Evaluator

Sorry, the evaluator is still being tested. You're welcome to use the beta version, and you can report any bugs via Gmail. We'll fix them as soon as possible!

Link to evaluator (https://github.com/coherent17/Optimizer-and-Legalizer-Co-optimization/blob/main/Evaluator)

```
$ ./Evaluator *.lg *.opt *_post.lg
```

Grading

There will be 3 public test cases and 1 hidden test case. You can get at least 20% * 0.7 = 14% if you are able to produce a legal result. The remaining 6% will depend on the quality and runtime of your solution. Another 20% comes from the report. As an engineer, not only is coding important, but proper and concise documentation of the algorithm is also crucial. Please try to demonstrate to the TA how great your algorithm is.

- Test Cases:
  - 3 public test cases.
  - 1 hidden test case.
- Grading Breakdown:

    - 14% for producing a legal result for each testcase.
    - 6% based on the solution's quality and runtime for each testcase.
    - 20% for the report.

Please note that for the public test cases, TA reserves right to change the testcase based on the feedback and difficulty! Please do check the version of your local testcase by:

```
$ git pull --dry-run | grep -q -v 'Already up-to-date.' && changed=1
```

- Bonus:
  This problem is new to both us (TAs) and the students. We have done our best to define and simplify it so that it can be solved within three weeks. However, we believe there may still be unexpected bugs. Please report any potential problems or issues to us. If the problem or bug is verified, you will receive one additional point for this lab!

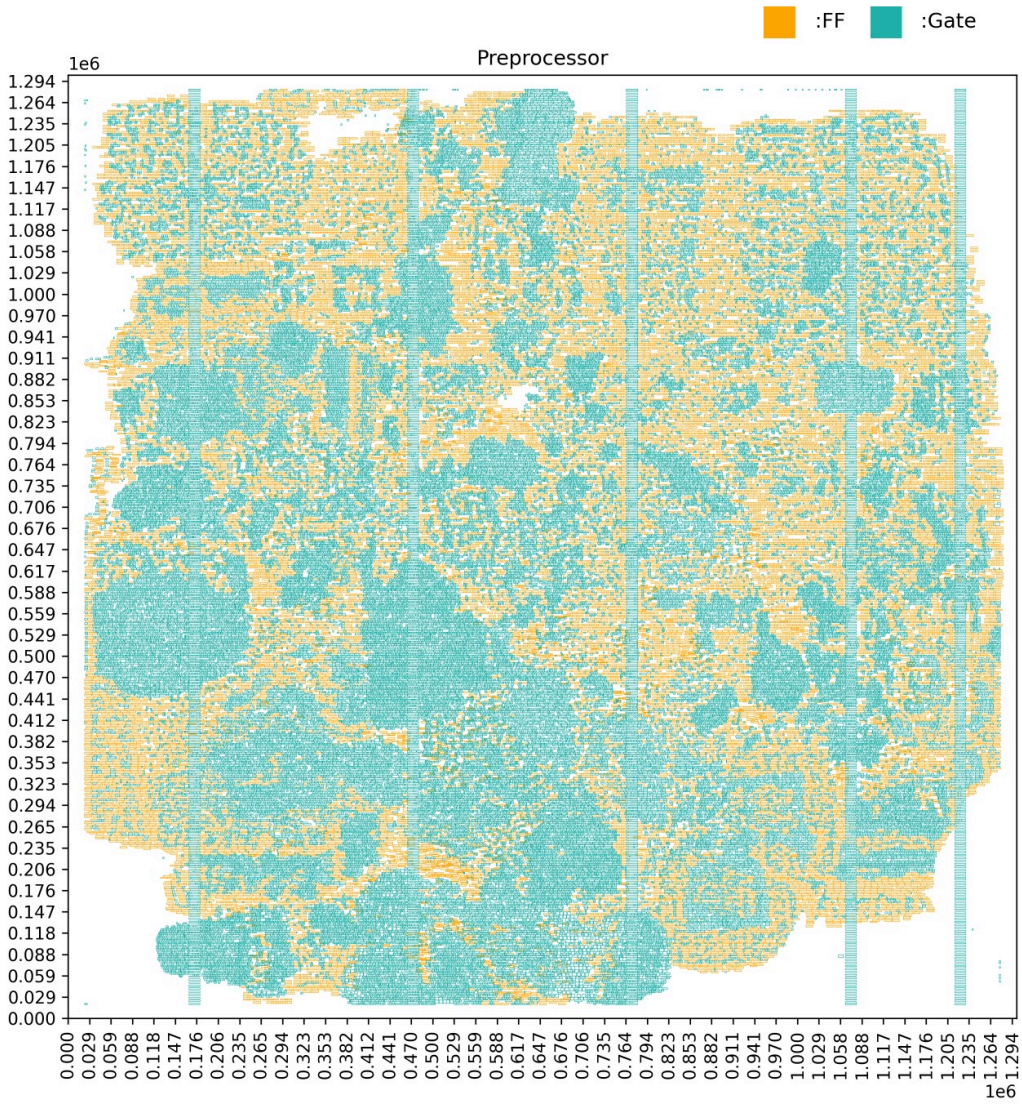
Disclaimer

- 1. TA reserves the right to change the rules mentioned above.
- 2. TA reserves the right to determine the final score.
- 3. TA reserves the right to modify this disclaimer.

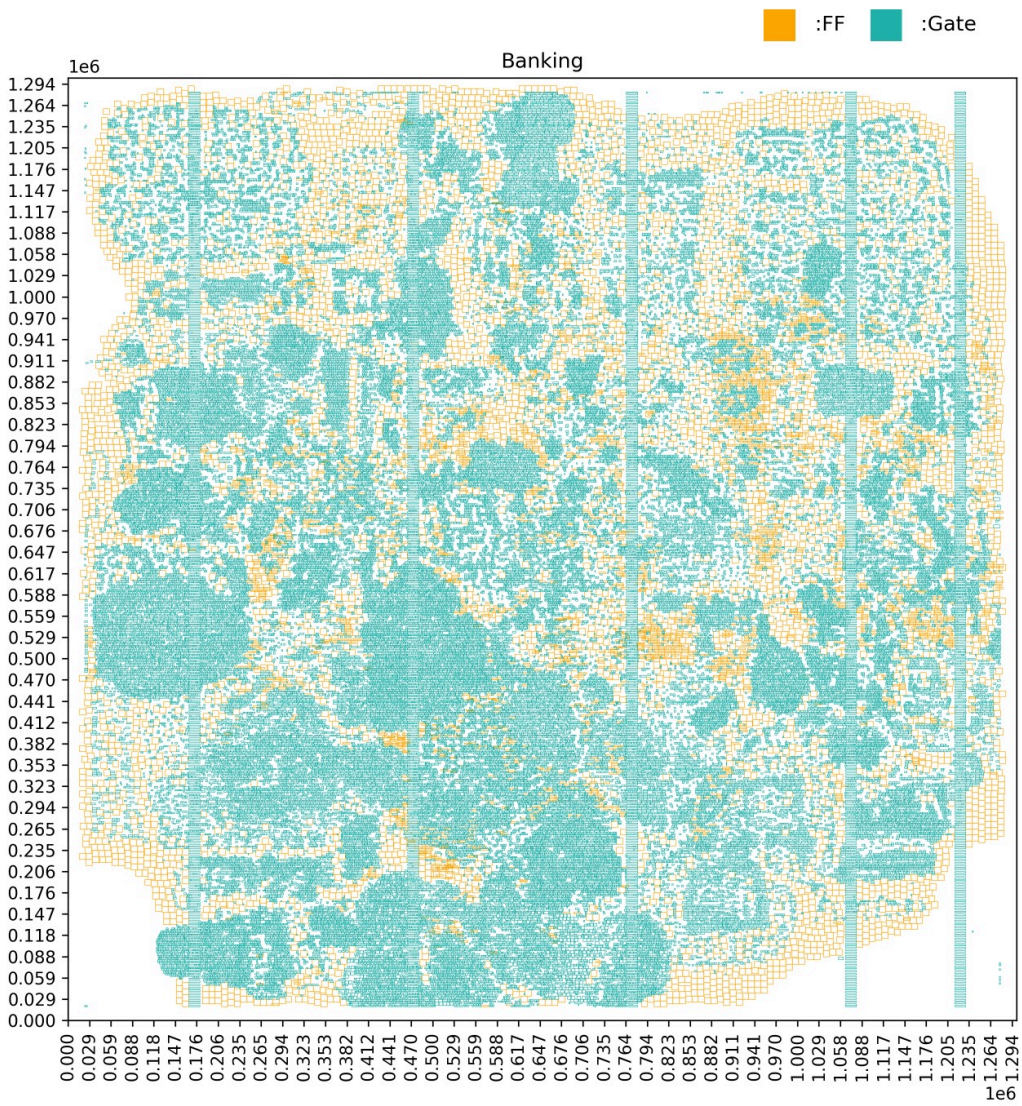
Visualizer

In EDA-related projects, it is good practice to visualize your results. Some well-known methods for doing this include using Python libraries like `matplotlib` or `seaborn`, and tools such as `gnuplot`. Visualizing your data helps in identifying trends, verifying correctness, and presenting your findings clearly. We might release a binary tool for plotting in the future, depending on available time...

1. Before Banking



2. After Banking



Some useful packages to plot efficiently:

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.collections import PatchCollection
```

Release

All test case updates and new versions of the evaluator will be uploaded through GitHub. We will provide notifications and make announcements in the E3. Please pull the latest test cases and evaluator to evaluate your binary.

Github Link: https://github.com/coherent17/Optimizer-and-Legalizer-Co-optimization (https://github.com/coherent17/Optimizer-and-Legalizer-Co-optimization)

```
$ git clone https://github.com/coherent17/Optimizer-and-Legalizer-Co-optimization
$ git pull
```

Reference

- Segment Tree (https://en.wikipedia.org/wiki/Segment_tree)
- Rtree (https://en.wikipedia.org/wiki/R-tree)
- Corner Stitch (https://www2.eecs.berkeley.edu/Pubs/TechRpts/1983/CSD-83-114.pdf)
- boost Interval (https://www.boost.org/doc/libs/1_86_0/libs/numeric/interval/doc/interval.htm)
- Tetris Legalization [Hill, Patent 2002] (https://patentimages.storage.googleapis.com/0d/17/0f/8b9dbb9343b70f/US6370673.pdf)
- Abacus Legalization [Spindler et al., ISPD'08] (https://dl.acm.org/doi/pdf/10.1145/1353629.1353640)
- Legalization Algorithm for Multiple-Row Height Standard Cell Design [Chow et al., DAC'16, best paper nominee at DAC'16] (https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7544326)
- An Effective Legalization Algorithm for Mixed-Cell-Height Standard Cells [C.-H. Wang] (https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7858364)
- FastDP [Pan, 2005] (https://www.researchgate.net/publication/221626852_An_efficient_and_effective_detailed_placement_algorithm)

Note that third-party placers or legalizers are not allowed in this project. You must implement the local legalizer yourself. However, data structures or algorithm-based packages are welcome. If you're unsure whether something is allowed, please contact the TA. Cheating or plagiarism will result in a score of 0 for this lab.

Q&A

This is the placeholder to discuss with the students.

Template format

- Q1:
  This is the question?

  ▶ 詳細資料

Contacts

- 1. TA1: Coherent, mnb51817@gmail.com (mailto:mnb51817@gmail.com)
- 2. TA2: Claire, m0970157475@gmail.com (mailto:m0970157475@gmail.com)

Please note that you should send your requests or questions to both TAs; otherwise, the TAs will not reply. Additionally, the TAs will update all Q&A in the session above to ensure fairness to all students. We will try to answer your questions within one day. Please be patient. Thank you!